DCRM — Dynamic Coordination Runtime Module

Parent Standard: Orchestration Governance Layer (OGL™)
Category: AI & Interpretation
Subcategory: Dynamic Coordination Runtime
Type: Orchestration Governance Module
Version: 1.0
Status: Canonical · Open Module
Effective Date: 8 May 2026
Compatibility: OOF® Methodology OS™ · OGL™ · **CLIA®** · **CML** · **RIS** · **INTEGROS®** · **EVIP®** · **Harness Architectures**
Authority: OOF®
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition

Dynamic Coordination Runtime Module defines the governance conditions under which orchestrated autonomous systems adapt coordination, change workflow state, reassign participation, reroute execution, or modify runtime behavior during live operation without losing structural validity, auditability, or control.

A system satisfies DCRM only if:

- runtime coordination changes occur under defined conditions
- dynamic reconfiguration remains bounded by governed logic
- participating agents, tools, or execution paths remain traceable during change
- adaptive coordination does not bypass authority, integrity, runtime, or ethical restrictions
- live orchestration change is restricted when runtime governance conditions are insufficient

A system that changes coordination behavior during execution without governed runtime control does not satisfy DCRM.

Module Function

DCRM defines the runtime-governance boundary of adaptive orchestration.

It ensures that dynamic coordination is not treated as automatically valid simply because it improves speed, flexibility, or task completion.

The module applies wherever multi-agent systems change workflow participation, execution order, routing logic, or operational coordination during live runtime conditions.

Minimum Implementation Framework (MIF)

Step 1 — Define Dynamic Change Conditions

The organization must define what kinds of runtime coordination changes are allowed.

Minimum requirement:

- valid dynamic changes are explicit
- prohibited runtime modifications are identifiable
- coordination change is not left open-ended

Step 2 — Define Trigger Logic

The system must define what triggers dynamic coordination change.

Minimum requirement:

- trigger conditions are explicit
- runtime adaptation is not treated as spontaneous or unexplained
- hidden trigger logic is excluded from valid coordination change

---

Step 3 — Define Reconfiguration Boundaries

The system must define how agents, tools, paths, or workflow states may change during operation.

Minimum requirement:

- reconfiguration boundaries are explicit
- allowed substitutions, rerouting, or reassignment remain bounded
- dynamic change does not silently expand orchestration scope

---

Step 4 — Define Runtime Validation Conditions

The system must define how changed coordination remains valid under live runtime conditions.

Minimum requirement:

- runtime validation logic is explicit
- changed execution structure is checked before continued reliance where required
- adaptation does not continue under unvalidated coordination change

---

Step 5 — Preserve Traceable Change Path

The system must preserve visibility of what changed, when, why, and under which runtime condition.

Minimum requirement:

- dynamic change path is traceable
- prior and current coordination states are distinguishable
- later review can reconstruct runtime adaptation history

Step 6 — Restrict Invalid Runtime Adaptation

The system must not be treated as valid if live coordination changes occur without governed boundaries, traceable justification, or runtime validation.

Minimum requirement:

- invalid dynamic coordination conditions are identifiable
- unbounded runtime adaptation is blocked
- flexibility does not override orchestration validity

Use Case 1 — Adaptive Multi-Agent Runtime Environment

Scenario

A multi-agent system dynamically changes which agents participate in execution based on task state, availability, intermediate results, or runtime pressure.

Application

DCRM is used to ensure:

- runtime change conditions are explicit
- trigger logic is governed
- reassignment remains bounded
- adaptation history remains traceable

Result

The organization gains a stronger basis for governing live coordination change without allowing dynamic flexibility to become unbounded orchestration behavior.

Use Case 2 — Autonomous Workflow Reconfiguration During Operation

Scenario

A runtime environment reroutes execution, inserts alternate paths, or changes coordination sequence while a workflow is already active.

Application

DCRM is used to ensure:

- reconfiguration boundaries are explicit
- live adaptation remains validated
- changed workflow state does not silently bypass governance conditions
- runtime coordination remains auditable and structurally controlled

---

Result

The organization gains a structurally governed runtime adaptation layer instead of relying on emergent coordination change as if it were automatically valid.

Canonical Closing Statement

If coordination changes during runtime without governed boundaries and traceable validation, orchestration integrity has not been achieved.

Related Documents

→ Orchestration Governance Layer (OGL™)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

https://originopenfoundation.org/